# Grado en Ingeniería Informática
# Algorítmica - Prácticas

**Cristina Zuheros Montes**

Departamento de Ciencias de la Computación
Universidad de Granada

Mayo, 2026




UNIVERSIDAD
DE GRANADA

Este documento está protegido por la Ley de Propiedad Intelectual (Real Decreto Legislativo 1/1996, de 12 de abril).

Queda expresamente prohibido su uso o distribución sin autorización del autor.

# Práctica 5

Programación dinámica.

# ¿Qué es la Programación Dinámica (PD)?

Técnica de diseño de algoritmos para resolver problemas que pueden descomponerse en subproblemas más pequeños, cuyas soluciones se combinan para obtener la solución del problema original.

- Parecido a Divide y Vencerás (DyV) pero:
  - DyV se utiliza cuando los subproblemas son independientes.
  - PD se utiliza cuando los problemas se solapan.

- Parecido a Greedy (ambos resuelven por etapas) pero:
  - Greedy toma una decisión local y no vuelve atrás.
  - PD considera todas las alternativas y combina soluciones de subproblemas para garantizar el óptimo.



UNIVERSIDAD DE GRANADA

# Aplicación

PD se aplica cuando el problema cumple dos propiedades clave:

- Subproblemas solapados: al plantear la solución recursiva, un mismo problema se resuelve más de una vez.
- Subestructura óptima: la solución óptima del problema puede construirse a partir de soluciones óptimas de subproblemas.
- Idea PD: Resolver cada subproblema una sola vez, almacenar su resultado en una tabla y reutilizarlo cuando vuelva a aparecer, evitando así cálculos.



UNIVERSIDAD DE GRANADA

# Ejemplo: Fibonacci

Para obtener el valor de Fibonacci de un número $n$ tenemos la siguiente formulación recursiva:

$$F(n) = \begin{cases} 1, & \text{si } n < 2, \\ F(n-1) + F(n-2), & \text{si } n \geq 2. \end{cases}$$

## Técnica DyV

```
0  int Fibonacci(int n) {
1   if (n < 2)
2    return 1;
3   else
4    return Fibonacci(n - 1) +
5           Fibonacci(n - 2);
6  }
```

## Técnica PD

```
0  vector<int> V = {0, 1};
1
2  int Fibonacci(int n) {
3   if (n<V.size()) return V[n];
4   else{
5    for(int i = V.size(); i <= n; ++i){
6      V.push_back(V[i-1]+V[i-2]);
7    }
8    return V[n];
9   }
10 }
```

# Principio de optimalidad

Antes de aplicar PD, debemos comprobar si se cumple el principio de optimalidad de Bellman.

**Principio de Optimalidad de Bellman (POB)**
Una secuencia óptima de decisiones que resuelve un problema debe cumplir la propiedad de que cualquier subsecuencia de decisiones debe ser también óptima respecto al subproblema que resuelve.




UNIVERSIDAD
DE GRANADA

# Esquema general

Para resolver un problema mediante PD, se realizan los siguientes pasos:

1. Comprobar que su cumple el POB.
2. Definir los subproblemas.
3. Definir una ecuación de recurrencia, especificando los casos base.
4. Rellenar una tabla de resultados intermedios.
5. Reconstruir la solución óptima si es necesario.



UNIVERSIDAD DE GRANADA

# Ejemplo

El problema del cambio de monedas.

# Planteamiento del problema

Dado un conjunto de $n$ monedas $(v_1, v_2, \ldots, v_n)$, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.

Datos del problema: Se dispone de monedas de valor 2, 3 y 7 y se desea devolver (o alcanzar) la cantidad $M = 8$.

- $M = 8$ ; $n = 3$
- $v_1 = 2; v_2 = 3; v_3 = 7$
- Aplicando el algoritmo greedy que elige en cada paso la moneda de mayor valor posible no obtenemos solución.
- ¿Se puede aplicar un enfoque de programación dinámica?

# Principio de optimalidad (POB)- Planteamiento

Sea $x_1, \ldots, x_n$ una solución óptima para el problema de cambiar $M$, donde $x_i$ representa el número de monedas de tipo $i$, con valor $v_i$. Por tanto:

$$\sum_{i=1}^{n} x_i v_i = M.$$

Sin pérdida de generalidad, podemos demostrar que, si fijamos el número de monedas de tipo $n$, entonces la parte restante

$$x_1, \ldots, x_{n-1}$$

debe ser óptima para el problema de cambiar $M - x_n v_n$ usando únicamente los primeros $n-1$ tipos de monedas.



UNIVERSIDAD DE GRANADA

# POB - Demostración I

Supongamos, por contradicción, que $x_1, \ldots, x_{n-1}$ no es óptima para cambiar $M - x_n v_n$ usando los primeros $n-1$ tipos de monedas. Entonces existe otra solución $y_1, \ldots, y_{n-1}$ para ese subproblema que usa menos monedas, es decir:

$$\sum_{i=1}^{n-1} y_i v_i = M - x_n v_n \qquad \text{y} \qquad \sum_{i=1}^{n-1} y_i < \sum_{i=1}^{n-1} x_i.$$

Ahora construimos una solución para el problema original manteniendo las mismas $x_n$ monedas de tipo $n$. Definimos $y_n = x_n$. Entonces,

$$\sum_{i=1}^{n} y_i v_i = \sum_{i=1}^{n-1} y_i v_i + y_n v_n = (M - x_n v_n) + x_n v_n = M.$$

Por tanto, $y_1, \ldots, y_n$ es una solución válida para cambiar $M$.



UNIVERSIDAD DE GRANADA

# POB - Demostración II

El número de monedas que tiene la solución $y_1, \ldots, y_n$ es:

$$\sum_{i=1}^{n} y_i = \sum_{i=1}^{n-1} y_i + y_n = \sum_{i=1}^{n-1} y_i + x_n.$$

Por hipótesis teníamos:

$$\sum_{i=1}^{n-1} y_i < \sum_{i=1}^{n-1} x_i.$$

Sumando $x_n$ a ambos lados de la desigualdad, tenemos:

$$\sum_{i=1}^{n} y_i < \sum_{i=1}^{n-1} x_i + x_n = \sum_{i=1}^{n} x_i.$$



UNIVERSIDAD DE GRANADA

# POB - Demostración III

Hemos construido una solución $y_1, \ldots, y_n$ para el problema de cambiar $M$ y hemos visto que usa menos monedas que la solución $x_1, \ldots, x_n$.

Esto contradice que $x_1, \ldots, x_n$ era una solución óptima para cambiar $M$.

Por tanto, la suposición inicial era falsa. En consecuencia,

$$x_1, \ldots, x_{n-1}$$

sí debe ser una solución óptima para cambiar $M - x_n v_n$ usando únicamente los primeros $n-1$ tipos de monedas.

Luego se cumple el principio de optimalidad. → Podemos aplicar programación dinámica.




UNIVERSIDAD DE GRANADA

# Definición de subproblemas

IDEA: Interpretar el problema como una toma de decisiones entre coger o no coger una moneda de tipo $i$.

Sea $C(i,j)$ el número mínimo de monedas necesarias para obtener la cantidad $j$ usando monedas de tipo $1,\ldots,i$, cuyos valores son $v_1,\ldots,v_i$.

En cada subproblema $C(i,j)$ hay dos alternativas:

- **No** usar ninguna moneda del tipo $i$: solución = $C(i-1,j)$.
- **Sí** usar al menos una moneda del tipo $i$: solución = $1 + C(i,j-v_i)$. (necesario $v_i \leq j$)

$$\Rightarrow \quad C(i,j) = \begin{cases} C(i-1,j), & \text{si } j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & \text{si } v_i \leq j. \end{cases}$$



UNIVERSIDAD DE GRANADA

# Ecuación de recurrencia - I

$$C(i,j) = \begin{cases} C(i-1,j), & \text{si } j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & \text{si } v_i \leq j. \end{cases}$$

Esta ecuación presenta problemas cuando i=1. Además, si $j = 0$ no harán falta modenas. $\Longrightarrow$

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 1,\ j < v_1, \\ 1 + C(1,j-v_1), & i = 1,\ v_i \leq j, \\ C(i-1,j), & i > 1,\ j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & \text{en otro caso.} \end{cases}$$



UNIVERSIDAD DE GRANADA

# Ecuación de recurrencia - II

Se puede simplificar la ecuación si incorporamos $i = 0$ a la tabla de resultados. Para $i = 0$, harán falta una cantidad infinita de monedas para cualquier $j > 0$.

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$



UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i, j - v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | | | | | | | | | |
| **2** | | | | | | | | | |
| **3** | | | | | | | | | |
| **7** | | | | | | | | | |

$i = 0, v_0 = \varnothing$

$i \in \{1, 2, 3\}$ $(v_1 = 2; v_2 = 3; v_3 = 7)$

$j \in \{0, \ldots, 8\}$

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | | | | | | | | |
| **2** | 0 | | | | | | | | |
| **3** | 0 | | | | | | | | |
| **7** | 0 | | | | | | | | |

Caso 1.
Para devolver 0, el resultado es 0 monedas.

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | | | | | | | | |
| **3** | 0 | | | | | | | | |
| **7** | 0 | | | | | | | | |

Caso 2.
No se puede devolver una cantidad $j > 0$ sin modenas.

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | | | | | | | |
| **3** | 0 | $\infty$ | | | | | | | |
| **7** | 0 | $\infty$ | | | | | | | |

Caso 3.
No se puede devolver el valor 1 con modenas de 2 (tampoco de 3 ni de 7).
Propagamos el valor de la fila previa.



UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | | | | | | |
| **3** | 0 | $\infty$ | | | | | | | |
| **7** | 0 | $\infty$ | | | | | | | |

Caso 4.
Se comparan $C(0,2)$ y $1 + C(2,0)$

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | | | | | | |
| **3** | 0 | $\infty$ | 1 | | | | | | |
| **7** | 0 | $\infty$ | 1 | | | | | | |

Caso 3.
No se puede devolver el valor 2 con modenas de 3 (tampoco de 7).
Propagamos el valor de la fila previa.

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | $\infty$ | | | | | |
| **3** | 0 | $\infty$ | 1 | | | | | | |
| **7** | 0 | $\infty$ | 1 | | | | | | |

Caso 4.
Se comparan $C(0,2)$ y $1 + C(2,0)$

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | $\infty$ | | | | | |
| **3** | 0 | $\infty$ | 1 | 1 | | | | | |
| **7** | 0 | $\infty$ | 1 | | | | | | |

Caso 4.
Se comparan $C(2,3)$ y $1 + C(3,0)$

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | $\infty$ | | | | | |
| **3** | 0 | $\infty$ | 1 | 1 | | | | | |
| **7** | 0 | $\infty$ | 1 | 1 | | | | | |

Caso 3.
No se puede devolver el valor 3 con modenas de 7.
Propagamos el valor de la fila previa.

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | $\infty$ | 2 | | | | |
| **3** | 0 | $\infty$ | 1 | 1 | | | | | |
| **7** | 0 | $\infty$ | 1 | 1 | | | | | |

Caso 4.
Se comparan $C(0,4)$ y $1 + C(2,2)$

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | $\infty$ | 2 | | | | |
| **3** | 0 | $\infty$ | 1 | 1 | 2 | | | | |
| **7** | 0 | $\infty$ | 1 | 1 | | | | | |

Caso 4.
Se comparan $C(1,4)$ y $1 + C(2,2)$

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | $\infty$ | 2 | | | | |
| **3** | 0 | $\infty$ | 1 | 1 | 2 | | | | |
| **7** | 0 | $\infty$ | 1 | 1 | 2 | | | | |

Caso 3.
No se puede devolver el valor 4 con modenas de 7.
Propagamos el valor de la fila previa.

UNIVERSIDAD DE GRANADA

# Construcción de la tabla

$$C(i,j) = \begin{cases} 0, & j = 0, \\ \infty, & i = 0, j \neq 0, \\ C(i-1,j), & j < v_i, \\ \min\big(C(i-1,j),\ 1 + C(i,j-v_i)\big), & v_i \leq j. \end{cases}$$

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| **2** | 0 | $\infty$ | 1 | $\infty$ | 2 | $\infty$ | 3 | $\infty$ | 4 |
| **3** | 0 | $\infty$ | 1 | 1 | 2 | 2 | 2 | 3 | 3 |
| **7** | 0 | $\infty$ | 1 | 1 | 2 | 2 | 2 | 1 | 3 |

UNIVERSIDAD DE GRANADA

# Reconstrucción del resultado

En la posición $(i, j)$:

- Si $C(i,j) = C(i-1,j)$: no se usa moneda $i$; ir a $(i-1,j)$.
- Si $C(i,j) = 1 + C(i, j-v_i)$: se usa moneda $i$; ir a $(i, j-v_i)$.

| | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| *0* | 0 | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |
| **2** | 0 | ∞ | 1 | ∞ | 2 | ∞ | 3 | ∞ | 4 |
| **3** | 0 | ∞ | 1 | 1 | 2 | 2 | 2 | 3 | 3 |
| **7** | 0 | ∞ | 1 | 1 | 2 | 2 | 2 | 1 | 3 |

Terminar cuando $j = 0$.

**Traza para $C(3,8) = 3$:**

1. $C(3,8) = C(2,8)$: no se usa $v_3 = 7$. Ir a $(2,8)$.
2. $C(2,8) \neq C(1,8)$: usar una moneda $v_2 = 3$. Ir a $(2,5)$.
3. $C(2,5) \neq C(1,5)$: usar una moneda $v_2 = 3$. Ir a $(2,2)$.
4. $C(2,2) = C(1,2)$: no se usa $v_2 = 3$. Ir a $(1,2)$.
5. $C(1,2) \neq C(0,2)$: usar una moneda $v_1 = 2$. Ir a $(1,0)$. Fin.

UNIVERSIDAD DE GRANADA